

CONTENTS:-

SL.NUMBER	TOPIC NAME	PAGE NUMBER
1	ABOUT DATA SET	01-02
2	VARIABLE INFORMATION OF DATA SET	02-04
3	TYPE OF MACHINE LEARNING USED IN THE DATASET	04-05
4	DATA PREPROCESSING	06-07
5	CODE	08-14
6	EXPLANATION OF CODE	14-17
7	RESULT	17-18
8	CONCLUSION	19-20
9	REFERENCE	

1. About Dataset

The dataset used in this assignment is the Steel Plates Faults dataset, obtained from the UCI Machine Learning Repository (ID: 198).

Property	Details
Dataset Name	Steel Plates Faults
UCI ID	198
Area	Physics and Chemistry
Task	Classification
Characteristics	Multivariate
Number of Instances	1941
Number of Features	27
Feature Types	Integer, Real
Missing Values	No
Year of Creation	2010
Creators	M Buscema, S Terzi, W Tastle
Dataset DOI	10.24432/C5J88N

The objective of this project is to classify steel plate defects into seven fault categories using machine learning techniques. This is a multiclass classification problem where each steel plate belongs to one of the following classes: Pastry, Z_Scratch, K_Scratch, Stains, Dirtiness, Bumps, and Other_Faults.

Fault Classes:

- Pastry - Surface defect resembling pastry-like marks.
- Z_Scratch - Scratch shaped like the letter Z.
- K_Scratch - Scratch with K-like pattern.
- Stains - Surface discoloration or stains.
- Dirtiness - Dirt contamination on the steel surface.
- Bumps - Raised surface irregularities.
- Other_Faults - Miscellaneous fault categories.

Dataset shape: Features (1941, 27) | Targets (1941, 7).

Class Distribution:

Fault Class	Number of Samples
Other_Faults	673
Bumps	402
K_Scratch	391
Z_Scratch	190

Pastry	158
Stains	72
Dirtiness	55
Total	1941

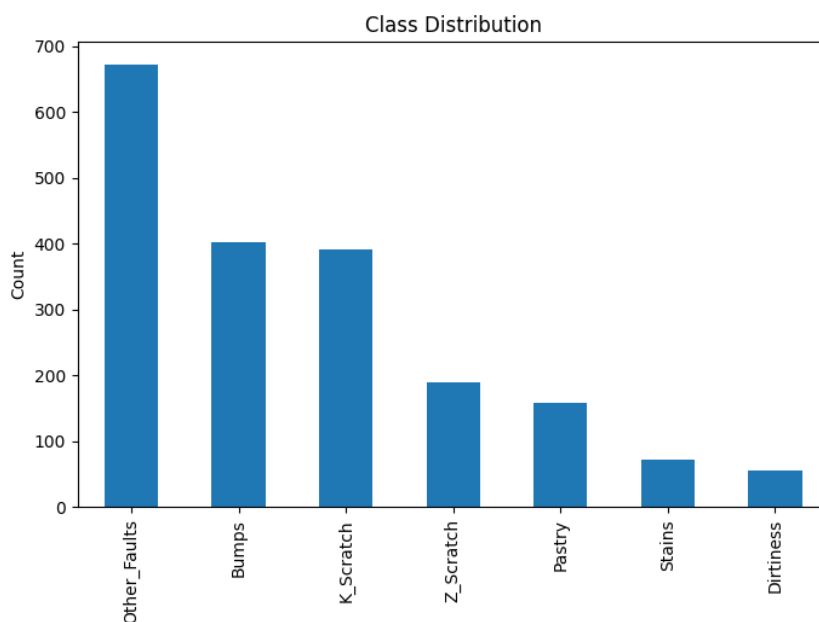


Figure 1: Class Distribution Bar Chart

The class distribution shows that the dataset is imbalanced. Other_Faults (673) is the majority class and Dirtiness (55) is the minority class.

2. Variable Information of Dataset

The dataset contains 27 independent (feature) variables and 7 dependent (target) variables describing geometric, luminosity, and positional characteristics of steel plate surface defects.

Variable Name	Role	Type
X_Minimum	Feature	Integer
X_Maximum	Feature	Integer
Y_Minimum	Feature	Integer
Y_Maximum	Feature	Integer
Pixels_Areas	Feature	Integer
X_Perimeter	Feature	Integer
Y_Perimeter	Feature	Integer
Sum_of_Luminosity	Feature	Integer
Minimum_of_Luminosity	Feature	Integer
Maximum_of_Luminosity	Feature	Integer
Length_of_Conveyer	Feature	Integer

TypeOfSteel_A300	Feature	Integer
TypeOfSteel_A400	Feature	Integer
Steel_Plate_Thickness	Feature	Integer
Edges_Index	Feature	Real
Empty_Index	Feature	Real
Square_Index	Feature	Real
Outside_X_Index	Feature	Real
Edges_X_Index	Feature	Real
Edges_Y_Index	Feature	Real
Outside_Global_Index	Feature	Real
LogOfAreas	Feature	Real
Log_X_Index	Feature	Real
Log_Y_Index	Feature	Real
Orientation_Index	Feature	Real
Luminosity_Index	Feature	Real
SigmoidOfAreas	Feature	Real
Pastry	Target	Integer
Z_Scratch	Target	Integer
K_Scratch	Target	Integer
Stains	Target	Integer
Dirtiness	Target	Integer
Bumps	Target	Integer
Other_Faults	Target	Integer

3. Type of Machine Learning Used in the Dataset

This project uses Supervised Learning - specifically, Multiclass Classification. Three machine learning algorithms were applied and compared:

3.1 K-Nearest Neighbors (KNN)

KNN classifies a data point based on the majority class of its K nearest neighbors. Used with `n_neighbors=5`. Requires feature scaling (StandardScaler) since it is a distance-based algorithm.

3.2 Decision Tree Classifier

A Decision Tree splits data based on feature values to make classification decisions. Can model nonlinear relationships and does not require feature scaling. Trained with `random_state=42`.

3.3 Random Forest Classifier

Random Forest builds multiple decision trees and uses majority voting for final prediction. Reduces overfitting and improves performance. Trained with `n_estimators=200` and `random_state=42`.

4. Data Preprocessing

4.1 Dataset Loading

The dataset was fetched using `fetch_ucirepo(id=198)`. Features (x) and targets (y) extracted as separate DataFrames.

4.2 Target Variable Conversion

The `idxmax()` function was used to convert the 7 binary target columns into a single multiclass string label. Each row has exactly one column with value 1, so `idxmax()` returns the fault class name.

4.3 Train-Test Split

80:20 split (`test_size=0.2`, `random_state=42`). Training: 1552 samples. Testing: 389 samples.

4.4 Feature Scaling

StandardScaler applied. Fit only on training data (`fit_transform`), then applied to test data (`transform`) to prevent data leakage. Required especially for KNN.

4.5 No Missing Values

The dataset has no missing values, so no imputation was required.

Correlation Heatmap:

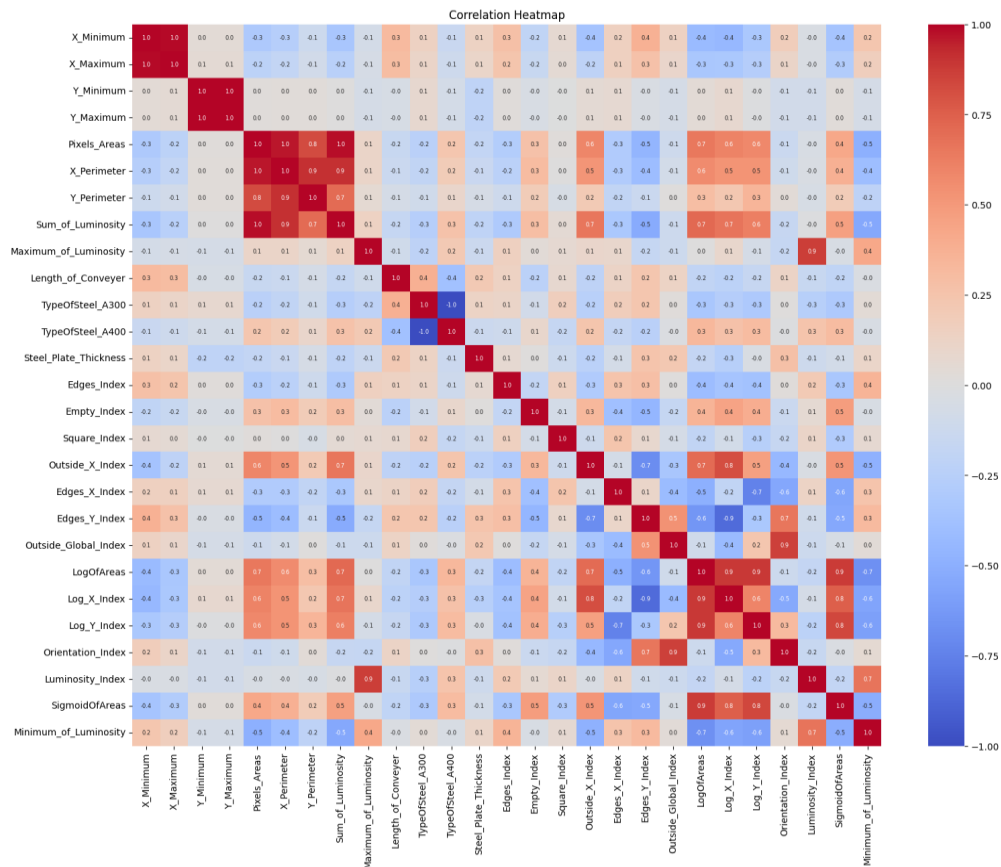


Figure 2: Correlation Heatmap of Features

Some geometric and luminosity features exhibit positive correlation. Most features show weak to moderate correlation, indicating diverse information useful for fault classification.

Feature Histograms:

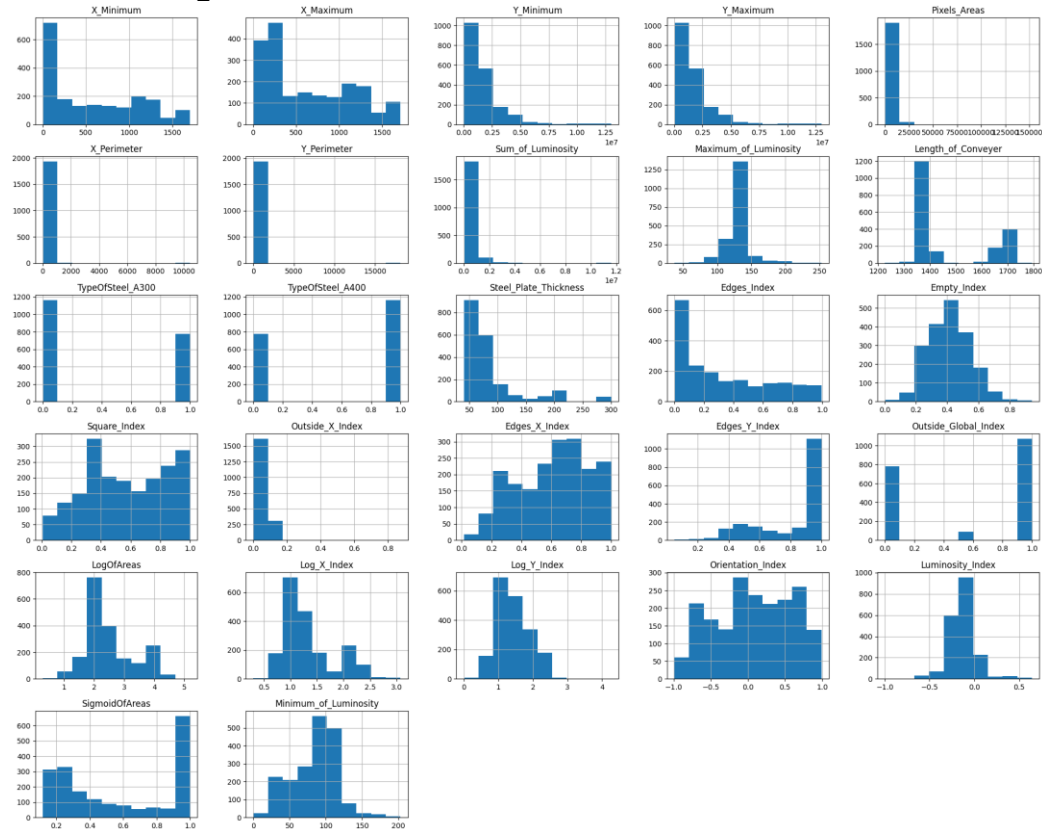


Figure 3: Histograms of All 27 Features

Boxplot:

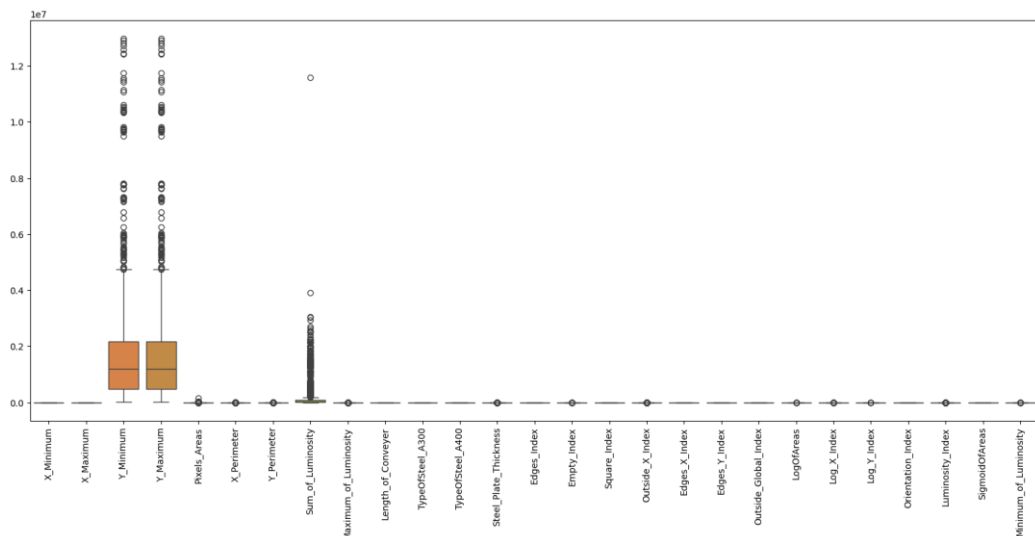


Figure 4: Boxplot of Features

The boxplot reveals outliers in several features. These outliers reflect real-world variability in manufacturing processes and are expected in industrial datasets.

5. Code

Step 1: Install Required Library

```
!pip install ucimlrepo
```

Step 2: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

Step 3: Load Dataset

```
from ucimlrepo import fetch_ucirepo

# fetch dataset
steel_plates_faults = fetch_ucirepo(id=198)

# data (as pandas dataframes)
x = steel_plates_faults.data.features
y = steel_plates_faults.data.targets

# metadata
print(steel_plates_faults.metadata)

# variable information
print(steel_plates_faults.variables)
```

Step 4: Check Shape

```
print(x.shape)
print(y.shape)
```

Output:

```
(1941, 27)
(1941, 7)
```

Step 5: Target Variable Conversion

```
y = y.idxmax(axis=1)
```

Step 6: View Unique Classes

```
print(y.unique())
```

Output:

```
['Pastry' 'Z_Scratch' 'K_Scratch' 'Stains' 'Dirtiness' 'Bumps'
 'Other_Faults']
```

Step 7: Class Value Counts

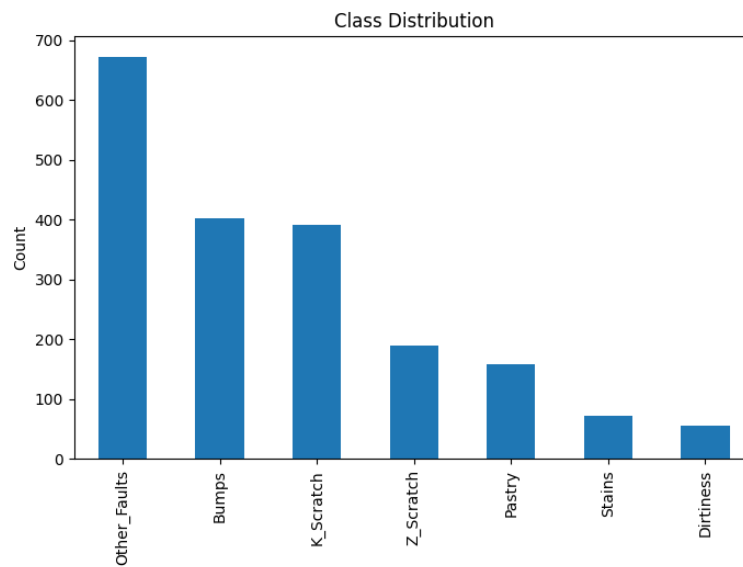
```
print(y.value_counts())
```

Output:

```
Other_Faults    673
Bumps          402
K_Scratch       391
Z_Scratch       190
Pastry          158
Stains          72
Dirtiness       55
Name: count, dtype: int64
```

Step 8: Class Distribution Bar Chart

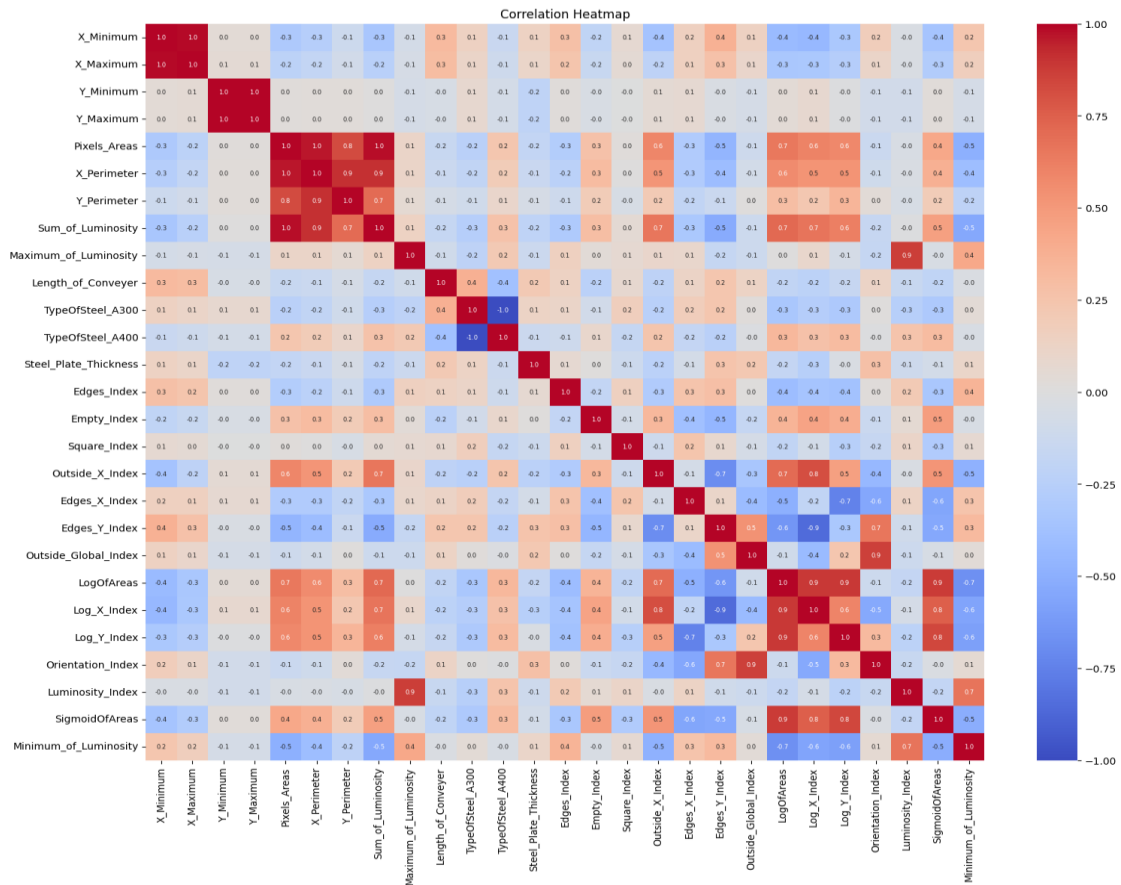
```
plt.figure(figsize=(8,5))
y.value_counts().plot(kind='bar')
plt.title("Class Distribution")
plt.ylabel("Count")
plt.show()
```



Output: Class Distribution Bar Chart

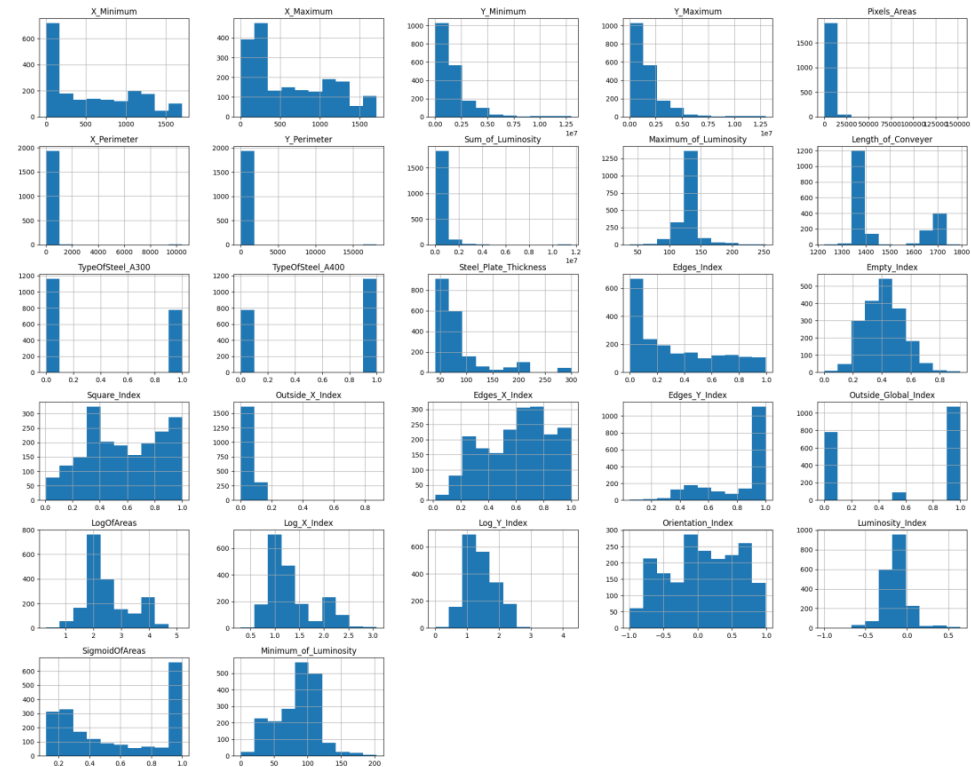
Step 9: Correlation Heatmap

```
plt.figure(figsize=(18,14))
sns.heatmap(
    x.corr(),
    annot=True,
    fmt=".1f",
    cmap="coolwarm",
    annot_kws={"size":6}
)
plt.title("Correlation Heatmap")
plt.show()
```



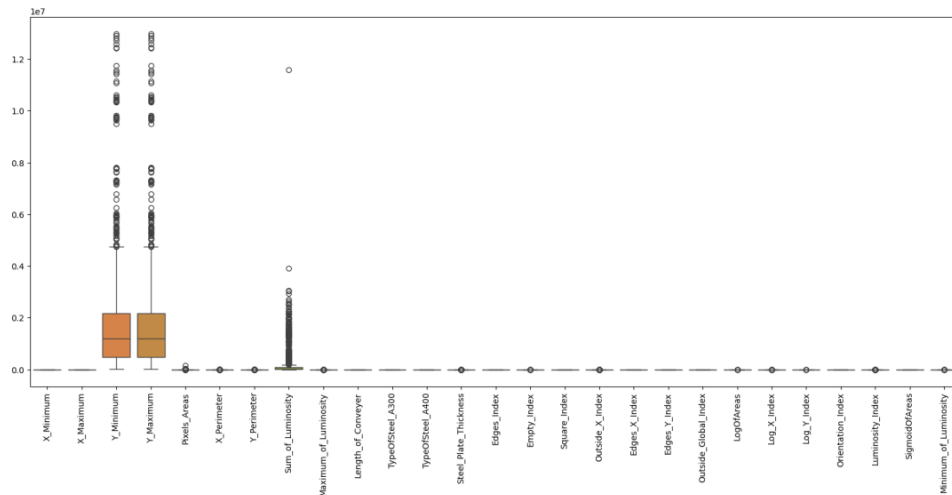
Step 10: Histograms

```
x.hist(figsize=(25,20))
plt.show()
```



Step 11: Boxplot

```
plt.figure(figsize=(20,8))
sns.boxplot(data=x)
plt.xticks(rotation=90)
plt.show()
```



Output: Boxplot of Features

Step 12: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 13: Feature Scaling

```
scaler = StandardScaler()
x_train = scaler.fit_transform(X_train)
x_test = scaler.transform(X_test)
```

Step 14: KNN Model

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train, y_train)

y_pred_knn = knn.predict(x_test)
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

Output:

KNN Accuracy: 0.7197943444730077

Step 15: Decision Tree Model

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

Output:

Decision Tree Accuracy: 0.7172236503856041

Step 16: Random Forest Model

```
rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)
rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Output:

```
Random Forest Accuracy: 0.7737789203084833
```

Step 17: Classification Reports

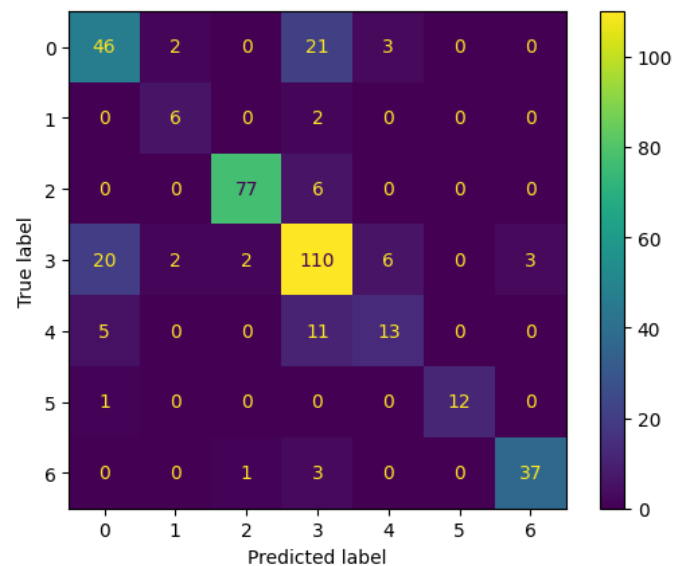
```
print("KNN Classification Report")
print(classification_report(y_test, y_pred_knn))

print("Decision Tree Classification Report")
print(classification_report(y_test, y_pred_dt))

print("Random Forest Classification Report")
print(classification_report(y_test, y_pred_rf))
```

Step 18: Confusion Matrix (Random Forest)

```
cm = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(cm)
disp.plot()
plt.show()
```



Output: Confusion Matrix - Random Forest

Step 19: Model Accuracy Comparison

```
results = {
    'KNN': accuracy_score(y_test, y_pred_knn),
    'Decision Tree': accuracy_score(y_test, y_pred_dt),
    'Random Forest': accuracy_score(y_test, y_pred_rf)
}
print(results)
```

Output:

```
{'KNN': 0.7197943444730077, 'Decision Tree': 0.7172236503856041, 'Random Forest': 0.7737789203084833}
```

Step 20: Feature Importance

```
importance = rf.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': x.columns,
    'Importance': importance
})
```

```
feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

print(feature_importance.head(10))
```

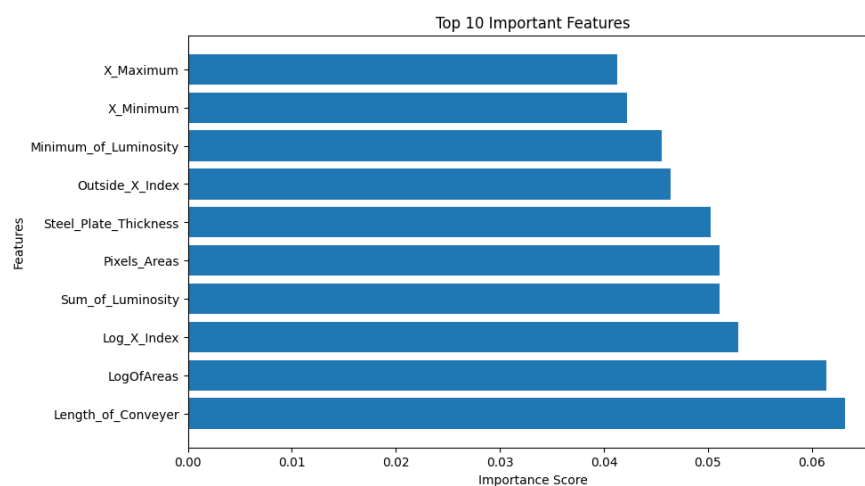
Output:

	Feature	Importance
9	Length_of_Conveyer	0.063170
20	LogOfAreas	0.061356
21	Log_X_Index	0.052915
7	Sum_of_Luminosity	0.051110
4	Pixels_Areas	0.051083
12	Steel_Plate_Thickness	0.050234
16	Outside_X_Index	0.046426
26	Minimum_of_Luminosity	0.045555
0	X_Minimum	0.042233
1	X_Maximum	0.041274

Step 21: Feature Importance Bar Chart

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.barh(
    feature_importance['Feature'][:10],
    feature_importance['Importance'][:10]
)
plt.title("Top 10 Important Features")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()
```



Output: Top 10 Important Features (Random Forest)

Step 22: Hyperparameter Tuning (GridSearchCV)

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20]
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5
```

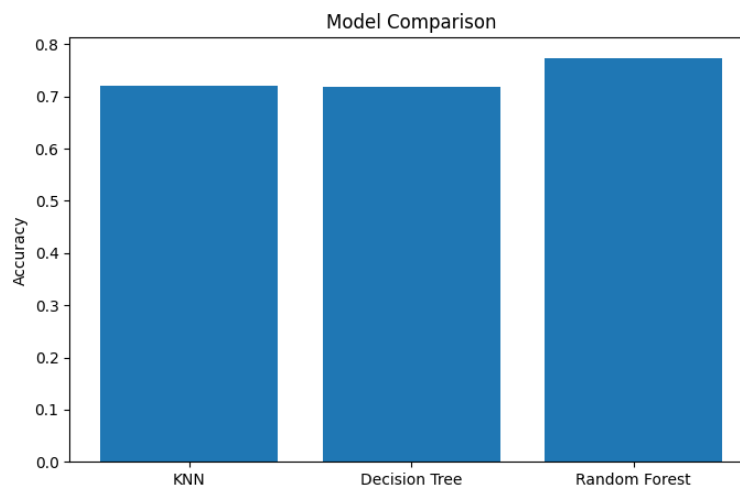
```
)  
  
grid.fit(X_train,y_train)  
  
print("Best Parameters:",grid.best_params_)  
print("Best Score:",grid.best_score_)
```

Output:

```
Best Parameters: {'max_depth': None, 'n_estimators': 200}  
Best Score: 0.7867026242091069
```

Step 23: Model Accuracy Bar Chart

```
plt.figure(figsize=(8,5))  
plt.bar(results.keys(), results.values())  
plt.ylabel("Accuracy")  
plt.show()
```



Output: Model Accuracy Comparison Bar Chart

6. Explanation of Code

Step 1: Install ucimlrepo

The ucimlrepo library is installed using pip. This library allows direct access to UCI Machine Learning Repository datasets in Python.

Step 2: Import Libraries

pandas and numpy are used for data manipulation. matplotlib and seaborn are used for visualization. sklearn provides machine learning models, preprocessing tools, and evaluation metrics.

Step 3: Load Dataset

The dataset is fetched using `fetch_ucirepo(id=198)`. Features (x) and targets (y) are stored as separate DataFrames.

Step 4: Check Shape

Shape of features is (1941, 27) and targets is (1941, 7), confirming 1941 samples with 27 features and 7 target columns.

Step 5: Target Variable Conversion

`idxmax()` is applied on the target DataFrame. Since each row has exactly one column with value 1, it returns the class name as a string label, converting 7 binary columns into a single multiclass variable.

Steps 6-8: Class Distribution

Unique class labels are printed confirming all 7 fault categories. `value_counts()` shows class imbalance. A bar chart visualizes the distribution. Other_Faults (673) is the majority class and Dirtiness (55) is the minority class.

Step 9: Correlation Heatmap

The heatmap shows correlation between all 27 features. Some geometric and luminosity features show higher correlation. Most features show weak to moderate correlation.

Steps 10-11: Histograms and Boxplot

Histograms show feature distributions. The boxplot reveals outliers in several features, which are expected in industrial steel plate data.

Step 12: Train-Test Split

80:20 split with `random_state=42` for reproducibility. 1552 samples for training and 389 for testing.

Step 13: Feature Scaling

`StandardScaler` normalizes features to zero mean and unit variance. Fitted on training data only to prevent data leakage, then applied to test data.

Steps 14-16: Model Training

KNN (`n_neighbors=5`) trained on scaled data. Decision Tree (`random_state=42`) and Random Forest (`n_estimators=200`, `random_state=42`) trained on original unscaled data. Predictions made on test data.

Step 17: Classification Reports

Reports show Precision, Recall, and F1-score per class. Important for imbalanced datasets where accuracy alone is insufficient.

Step 18: Confusion Matrix

Plotted for the Random Forest model. Diagonal values show correct predictions. Off-diagonal values show misclassifications between fault categories.

Steps 19-21: Accuracy Comparison and Feature Importance

Random Forest achieved the highest accuracy of 77.38%. Feature importance analysis identified Length_of_Conveyer and LogOfAreas as most influential features.

Step 22: Hyperparameter Tuning

GridSearchCV tested combinations of n_estimators and max_depth with 5-fold cross-validation. Best parameters: max_depth=None, n_estimators=200 with a score of 78.67%.

7. Result

Model Accuracy Comparison:

Model	Accuracy
K-Nearest Neighbors (KNN)	71.98%
Decision Tree	71.72%
Random Forest	77.38%

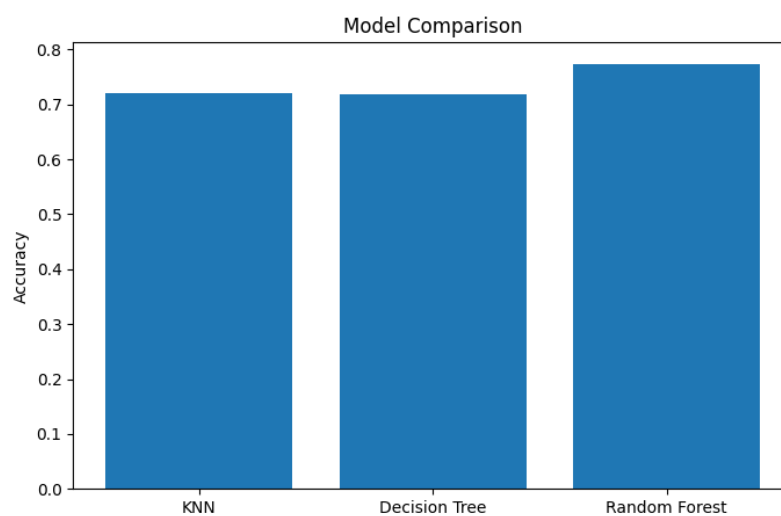


Figure 5: Model Accuracy Comparison

KNN Classification Report:

Class	Precision	Recall	F1-Score	Support
Bumps	0.56	0.68	0.61	72
Dirtiness	0.50	0.75	0.60	8
K_Scratch	0.93	0.93	0.93	83
Other_Faults	0.74	0.65	0.69	143
Pastry	0.32	0.21	0.25	29
Stains	0.86	0.92	0.89	13
Z_Scratch	0.79	0.90	0.84	41
Accuracy			0.72	389
Macro Avg	0.67	0.72	0.69	389
Weighted Avg	0.72	0.72	0.71	389

Decision Tree Classification Report:

Class	Precision	Recall	F1-Score	Support
-------	-----------	--------	----------	---------

Bumps	0.53	0.57	0.55	72
Dirtiness	0.50	0.75	0.60	8
K_Scratch	0.90	0.92	0.91	83
Other_Faults	0.74	0.66	0.70	143
Pastry	0.40	0.48	0.44	29
Stains	0.86	0.92	0.89	13
Z_Scratch	0.92	0.88	0.90	41
Accuracy			0.72	389
Macro Avg	0.69	0.74	0.71	389
Weighted Avg	0.73	0.72	0.72	389

Random Forest Classification Report:

Class	Precision	Recall	F1-Score	Support
Bumps	0.64	0.64	0.64	72
Dirtiness	0.60	0.75	0.67	8
K_Scratch	0.96	0.93	0.94	83
Other_Faults	0.72	0.77	0.74	143
Pastry	0.59	0.45	0.51	29
Stains	1.00	0.92	0.96	13
Z_Scratch	0.93	0.90	0.91	41
Accuracy			0.77	389
Macro Avg	0.78	0.77	0.77	389
Weighted Avg	0.78	0.77	0.77	389

Confusion Matrix (Random Forest):

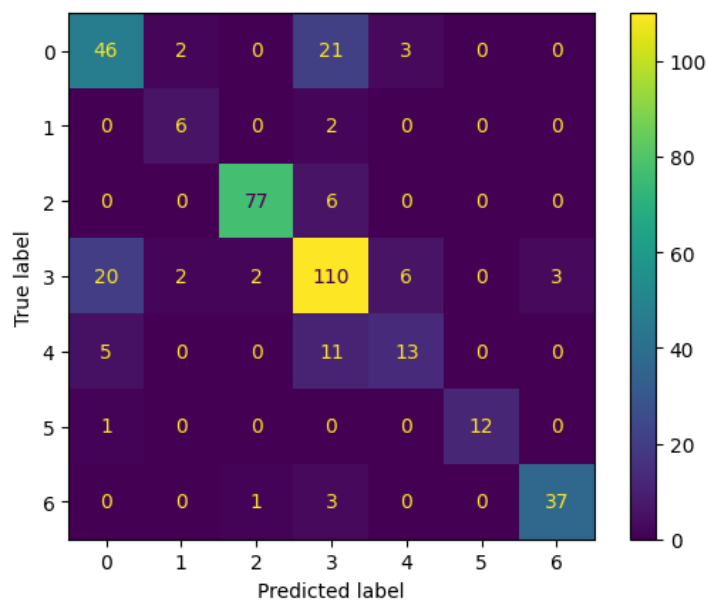


Figure 6: Confusion Matrix - Random Forest

Top 10 Important Features (Random Forest):

Rank	Feature	Importance Score
1	Length_of_Conveyer	0.063170
2	LogOfAreas	0.061356
3	Log_X_Index	0.052915
4	Sum_of_Luminosity	0.051110
5	Pixels_Areas	0.051083
6	Steel_Plate_Thickness	0.050234
7	Outside_X_Index	0.046426
8	Minimum_of_Luminosity	0.045555
9	X_Minimum	0.042233
10	X_Maximum	0.041274

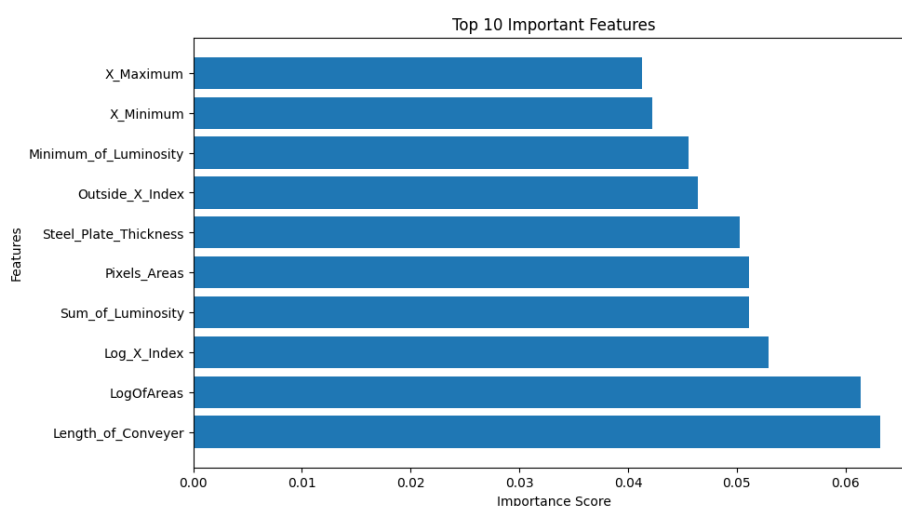


Figure 7: Feature Importance Bar Chart

Hyperparameter Tuning Result:

Parameter	Best Value
max_depth	None
n_estimators	200
Best Cross-Validation Score	78.67%

The Random Forest model achieved the best performance overall. K_Scratch and Stains had the highest F1-scores (0.94 and 0.96). Pastry had the lowest F1-score (0.51) due to limited samples and class overlap.

8. Conclusion

In this assignment, the Steel Plates Faults dataset from the UCI Machine Learning Repository was used to build a multiclass fault classification system. The dataset contains 1941 samples, 27 input features, and 7 fault categories.

Three machine learning models were applied and compared: K-Nearest Neighbors (KNN), Decision Tree, and Random Forest. The Random Forest model achieved the highest accuracy of 77.38%, followed by KNN (71.98%) and Decision Tree (71.72%).

From the classification reports, K_Scratch and Stains were classified with high F1-scores. Pastry showed the lowest performance due to limited training samples and overlap with other classes.

Feature importance analysis showed that Length_of_Conveyer, LogOfAreas, Log_X_Index, Sum_of_Luminosity, and Pixels_Areas are the most significant features for fault detection.

Hyperparameter tuning using GridSearchCV confirmed that max_depth=None and n_estimators=200 are the optimal parameters for the Random Forest model, achieving a cross-validation score of 78.67%.

The results show that Random Forest is the most suitable algorithm for this dataset. The dataset is imbalanced and this affects performance on minority classes such as Dirtiness and Pastry.

9. Reference

- UCI Machine Learning Repository - Steel Plates Faults Dataset:
<https://archive.ics.uci.edu/dataset/198/steel+plates+faults>
- M Buscema, S Terzi, W Tastle - Steel Plates Faults Dataset (2010). DOI:
10.24432/C5J88N
- scikit-learn Documentation - KNeighborsClassifier, DecisionTreeClassifier,
RandomForestClassifier, GridSearchCV: <https://scikit-learn.org/stable/>
- pandas Documentation - DataFrame.idxmax:
<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.idxmax.html>
- Google Colaboratory - Colab Notebooks: <https://colab.research.google.com>